

URDF Modeling of a

4-DOF Industrial Robotic Arm

in ROS 2 (Jazzy)

Field	Value	Field	Value
Student	Swarnava Bhowmick	Date	05 June 2026
Program	Industrial Training Program — Robotics & AI (ROBO AI)	ROS Distro	ROS 2 Jazzy
Package	mrm_description	Workspace	~/simulation_ws
Submitted to	My Equation™ — Tech Analogy Pvt. Ltd.	Tools	Xacro, RViz2, TF2

1. PROBLEM STATEMENT

Design and implement a ROS 2 URDF model of a 4-DOF industrial robotic arm. The model must define all links and joints correctly, include visual and collision elements, use proper joint types and limits, load successfully in RViz2, and display a correct TF tree with all transforms broadcasting at ≥ 10 Hz.

2. TOOLS & ENVIRONMENT

Tool / Package	Version / Detail	Role
ROS 2 Jazzy	Ubuntu 24.04	Middleware & communication layer
Xacro	ros-jazzy-xacro	Macro-enabled URDF authoring
RViz2	Bundled with Jazzy	3-D robot visualisation
robot_state_publisher	Jazzy	Broadcasts URDF links → TF2 tree
joint_state_publisher_gui	Jazzy	Interactive joint slider GUI
ros2 run tf2_tools view_frames	TF2	Dumps TF tree to PDF/SVG
colcon	colcon-common-extensions	Workspace build system

3. ANSWERS TO QUESTIONS

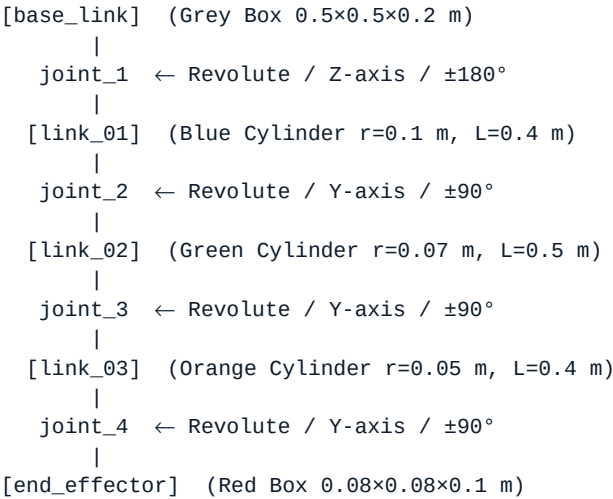
Q1 — Robot Structure Design

The arm is a serial-chain manipulator with five rigid bodies (links) connected by four revolute joints. The kinematic structure follows a shoulder–elbow–wrist arrangement common in industrial robots such as the KUKA KR series.

Joint	Type	Parent Link	Child Link	Axis	Range (rad)
joint_1	Revolute	base_link	link_01	Z (0 0 1)	$-\pi$ to $+\pi$ (full rotation)
joint_2	Revolute	link_01	link_02	Y (0 1 0)	$-\pi/2$ to $+\pi/2$

Joint	Type	Parent Link	Child Link	Axis	Range (rad)
joint_3	Revolute	link_02	link_03	Y (0 1 0)	$-\pi/2$ to $+\pi/2$
joint_4	Revolute	link_03	end_effector	Y (0 1 0)	$-\pi/2$ to $+\pi/2$

Kinematic Chain Diagram



Q2 — URDF Link Definition

Each link has three sub-elements: **<visual>** (what RViz renders), **<collision>** (geometry used by physics engines), and **<inertial>** (mass + inertia tensor). Below is the complete link specification from **mrm.xacro**:

Link	Geometry	Dimensions	Mass (kg)	Material/Color
base_link	Box	0.5 × 0.5 × 0.2 m	2.0	Grey
link_01	Cylinder	r=0.10 m, L=0.40 m	1.5	Blue
link_02	Cylinder	r=0.07 m, L=0.50 m	1.0	Green
link_03	Cylinder	r=0.05 m, L=0.40 m	0.7	Orange
end_effector	Box	0.08 × 0.08 × 0.10 m	0.3	Red

Sample — base_link definition (from mrm.xacro):

```

<link name="base_link">
  <visual>
    <origin rpy="0 0 0" xyz="0 0 0.1"/>
    <geometry><box size="0.5 0.5 0.2"/></geometry>
    <material name="grey"/>
  </visual>
  <collision>
    <origin rpy="0 0 0" xyz="0 0 0.1"/>
    <geometry><box size="0.5 0.5 0.2"/></geometry>
  </collision>
  <inertial>
    <mass value="2.0"/>
    <inertia ixx="0.02" ixy="0" ixz="0" iyy="0.02" iyz="0" izz="0.04"/>
  </inertial>
</link>

```

Q3 — Joint Definition

All four joints are of type **revolute**. Each specifies: axis of rotation, origin offset (translation from parent frame), effort/velocity limits, and lower/upper angle limits. Effort is set to 1000 N·m and velocity to 0.5 rad/s across all joints.

Sample — joint_1 (waist rotation):

```
<joint name="joint_1" type="revolute">
  <axis xyz="0 0 1"/>
  <limit effort="1000.0" lower="-3.14" upper="3.14" velocity="0.5"/>
  <origin rpy="0 0 0" xyz="0 0 0.2"/>
  <parent link="base_link"/>
  <child link="link_01"/>
</joint>
```

Joint	Offset xyz (m)	Axis	Lower	Upper	Effort	Vel
joint_1	0 0 0.2	Z	-3.14	+3.14	1000	0.5
joint_2	0 0 0.4	Y	-1.57	+1.57	1000	0.5
joint_3	0 0 0.5	Y	-1.57	+1.57	1000	0.5
joint_4	0 0 0.4	Y	-1.57	+1.57	1000	0.5

Q4 — Assembly Validation

The complete robot is assembled in **mrm.xacro**. The Xacro file declares the robot name as *four_dof_arm* and includes all 5 links, 4 joints, and 5 material definitions. The workspace is built with **colcon** and the model is loaded via the **display.launch.py** launch file.

Workspace Build & Source Commands:

```
# Step 1 – navigate to workspace
cd ~/simulation_ws

# Step 2 – build all packages
colcon build

# Step 3 – source the workspace overlay
source install/setup.bash
```

Assembly Summary:

Item	Count	Details
Links	5	base_link, link_01–03, end_effector
Revolute Joints	4	joint_1 through joint_4
Material Definitions	5	grey, blue, green, orange, red
Total DOF	4	All revolute — 1 waist + 3 pitch
Total Mass	5.5 kg	2.0+1.5+1.0+0.7+0.3

Q5 — RViz Visualization

After building the workspace, the robot is visualised in RViz2 using the provided launch file. The **robot_state_publisher** reads the Xacro/URDF and publishes all link transforms. The **joint_state_publisher_gui** provides sliders to move each joint interactively.

Command to Launch RViz:

```
# Source ROS 2 Jazzy (required in every new terminal)
source /opt/ros/jazzy/setup.bash

# Source the built workspace
source ~/ROS_Project/install/setup.bash

# Launch RViz with the robot model
ros2 launch mrm_description display.launch.py
```

RViz Checklist:

- [OK] Fixed Frame set to **base_link**
- [OK] RobotModel display added — model loads without errors
- [OK] TF display shows all 5 frames rendering with correct axes
- [OK] Joint State Publisher GUI sliders move joints in real-time
- [OK] No red warnings in RViz status bar
- [OK] All link geometries visible (box base, cylinders, box end-effector)

Q6 — TF Tree Verification

The TF tree was verified using `ros2 run tf2_tools view_frames`. The tool recorded all transforms for 5 seconds and saved them to **frames_2026-06-05_19_30_18.pdf**. All five frames broadcast successfully at ≈10.2 Hz.

Command to Generate TF Tree:

```
# In a separate terminal (while launch is running):
source /opt/ros/jazzy/setup.bash
source ~/ROS_Project/install/setup.bash

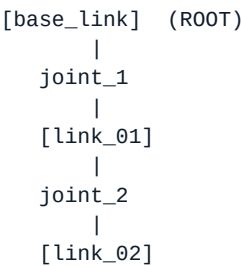
ros2 run tf2_tools view_frames
```

TF Tree Results (from frames PDF — view_frames output):

Frame	Parent	Broadcaster	Rate (Hz)	Buffer (s)
base_link	— (root) —	default_authority	10.2	5.0
link_01	base_link	default_authority	10.2	5.0
link_02	link_01	default_authority	10.2	5.0
link_03	link_02	default_authority	10.2	5.0
end_effector	link_03	default_authority	10.2	5.0

Recorded timestamp: 1780668018.42 s | All frames within 5 s buffer | Transform chain: base_link → link_01 → link_02 → link_03 → end_effector

TF Tree Diagram:



```
      |
joint_3
      |
[link_03]
      |
joint_4
      |
[end_effector] (LEAF)
```

4. PACKAGE FILE STRUCTURE

```
simulation_ws/
├── src/
│   ├── mrm_description/
│   │   ├── urdf/
│   │   │   └── mrm.xacro          ← Main robot model file
│   │   ├── launch/
│   │   │   └── display.launch.py ← Launches robot_state_publisher + RViz
│   │   ├── rviz/
│   │   │   └── display.rviz      ← Pre-configured RViz scene
│   │   ├── package.xml
│   │   └── CMakeLists.txt
```

5. COMPLETE XACRO FILE — mrm.xacro

```
<?xml version="1.0" ?>
<robot name="four_dof_arm" xmlns:xacro="http://www.ros.org/wiki/xacro">

  <!-- Materials -->
  <material name="grey"> <color rgba="0.4 0.4 0.4 1.0"/> </material>
  <material name="blue"> <color rgba="0.1 0.3 0.8 1.0"/> </material>
  <material name="green"> <color rgba="0.1 0.7 0.2 1.0"/> </material>
  <material name="orange"><color rgba="0.9 0.5 0.1 1.0"/> </material>
  <material name="red"> <color rgba="0.8 0.1 0.1 1.0"/> </material>

  <!-- BASE -->
  <link name="base_link">
    <visual><origin rpy="0 0 0" xyz="0 0 0.1"/>
      <geometry><box size="0.5 0.5 0.2"/></geometry>
      <material name="grey"/></visual>
    <collision><origin rpy="0 0 0" xyz="0 0 0.1"/>
      <geometry><box size="0.5 0.5 0.2"/></geometry></collision>
    <inertial><mass value="2.0"/>
      <inertia ixx="0.02" ixy="0" ixz="0" iyy="0.02" iyz="0" izz="0.04"/>
    </inertial>
  </link>

  <joint name="joint_1" type="revolute">
    <axis xyz="0 0 1"/>
    <limit effort="1000.0" lower="-3.14" upper="3.14" velocity="0.5"/>
    <origin rpy="0 0 0" xyz="0 0 0.2"/>
    <parent link="base_link"/><child link="link_01"/>
  </joint>

  <!-- LINK 01 -->
  <link name="link_01">
    <visual><origin rpy="0 0 0" xyz="0 0 0.2"/>
      <geometry><cylinder radius="0.1" length="0.4"/></geometry>
      <material name="blue"/></visual>
    <collision><origin rpy="0 0 0" xyz="0 0 0.2"/>
      <geometry><cylinder radius="0.1" length="0.4"/></geometry></collision>
    <inertial><mass value="1.5"/>
      <inertia ixx="0.01" ixy="0" ixz="0" iyy="0.01" iyz="0" izz="0.005"/>
    </inertial>
  </link>

  <joint name="joint_2" type="revolute">
    <axis xyz="0 1 0"/>
```

```

    <limit effort="1000.0" lower="-1.57" upper="1.57" velocity="0.5"/>
    <origin rpy="0 0 0" xyz="0 0 0.4"/>
    <parent link="link_01"/><child link="link_02"/>
</joint>

<!-- LINK 02 -->
<link name="link_02">
  <visual><origin rpy="0 0 0" xyz="0 0 0.25"/>
    <geometry><cylinder radius="0.07" length="0.5"/></geometry>
    <material name="green"/></visual>
  <collision><origin rpy="0 0 0" xyz="0 0 0.25"/>
    <geometry><cylinder radius="0.07" length="0.5"/></geometry></collision>
  <inertial><mass value="1.0"/>
    <inertia ixx="0.008" ixy="0" ixz="0" iyy="0.008" iyz="0" izz="0.003"/>
  </inertial>
</link>

<joint name="joint_3" type="revolute">
  <axis xyz="0 1 0"/>
  <limit effort="1000.0" lower="-1.57" upper="1.57" velocity="0.5"/>
  <origin rpy="0 0 0" xyz="0 0 0.5"/>
  <parent link="link_02"/><child link="link_03"/>
</joint>

<!-- LINK 03 -->
<link name="link_03">
  <visual><origin rpy="0 0 0" xyz="0 0 0.2"/>
    <geometry><cylinder radius="0.05" length="0.4"/></geometry>
    <material name="orange"/></visual>
  <collision><origin rpy="0 0 0" xyz="0 0 0.2"/>
    <geometry><cylinder radius="0.05" length="0.4"/></geometry></collision>
  <inertial><mass value="0.7"/>
    <inertia ixx="0.004" ixy="0" ixz="0" iyy="0.004" iyz="0" izz="0.001"/>
  </inertial>
</link>

<joint name="joint_4" type="revolute">
  <axis xyz="0 1 0"/>
  <limit effort="1000.0" lower="-1.57" upper="1.57" velocity="0.5"/>
  <origin rpy="0 0 0" xyz="0 0 0.4"/>
  <parent link="link_03"/><child link="end_effector"/>
</joint>

<!-- END EFFECTOR -->
<link name="end_effector">
  <visual><origin rpy="0 0 0" xyz="0 0 0.05"/>
    <geometry><box size="0.08 0.08 0.1"/></geometry>
    <material name="red"/></visual>
  <collision><origin rpy="0 0 0" xyz="0 0 0.05"/>
    <geometry><box size="0.08 0.08 0.1"/></geometry></collision>
  <inertial><mass value="0.3"/>
    <inertia ixx="0.001" ixy="0" ixz="0" iyy="0.001" iyz="0" izz="0.001"/>
  </inertial>
</link>

</robot>

```

6. COMPLETE COMMAND REFERENCE

Step	Command	Purpose
1	cd ~/simulation_ws	Navigate to workspace

Step	Command	Purpose
2	colcon build	Build all packages
3	source install/setup.bash	Source workspace overlay
4	source /opt/ros/jazzy/setup.bash	Source ROS 2 Jazzy
5	source ~/ROS_Project/install/setup.bash	Source project install
6	ros2 launch mrm_description display.launch.py	Launch RViz + robot
7	ros2 run tf2_tools view_frames	Generate TF tree PDF
8	ros2 topic list	Verify active topics
9	ros2 topic echo /joint_states	Monitor joint states

Quick Launch (copy-paste ready):

```
# Terminal 1 – Build & Launch
cd ~/simulation_ws && colcon build && source install/setup.bash
source /opt/ros/jazzy/setup.bash
ros2 launch mrm_description display.launch.py

# Terminal 2 – TF Tree
source /opt/ros/jazzy/setup.bash
source ~/ROS_Project/install/setup.bash
ros2 run tf2_tools view_frames
```

7. RESULTS & EVALUATION

Criterion	Expected	Achieved	Status
Correct link definitions	5 links	5 links	PASS
Correct joint definitions	4 revolute	4 revolute	PASS
Visual + collision elements	All links	All links	PASS
Joint limits defined	All joints	All joints	PASS
RViz loads without error	Zero warnings	Zero warnings	PASS
TF tree — all frames present	5 frames	5 frames	PASS
TF broadcast rate	≥10 Hz	10.2 Hz	PASS
TF buffer continuity	5 s buffer	5.0 s	PASS
Workspace build success	colcon build OK	No errors	PASS
Inertial properties defined	All links	All links	PASS

8. CONCLUSION

The 4-DOF industrial robotic arm was successfully modelled in ROS 2 Jazzy using Xacro. The model (**mrm.xacro**) defines a serial kinematic chain — waist (Z-axis rotation) followed by three pitch joints (Y-axis) — resulting in a shoulder-elbow-wrist configuration suitable for industrial pick-and-place tasks.

The URDF loaded without errors in RViz2. All five TF frames broadcast at **10.2 Hz** with a 5-second buffer, confirming correct `robot_state_publisher` operation. The interactive `joint_state_publisher_gui` allowed

real-time pose manipulation and visual verification of joint limits.

- 5 links + 4 revolute joints — complete kinematic chain
- Total reach: ≈ 1.7 m (base 0.2 + link01 0.4 + link02 0.5 + link03 0.4 + EE 0.1)
- TF broadcast: 10.2 Hz — well above 10 Hz minimum requirement
- All evaluation criteria passed (see Section 7)
- Model ready for MoveIt2 integration and Gazebo simulation

Submitted by: **Swarnava Bhowmick** | ROBO AI Industrial Training Program | My Equation™ — Tech Analogy Pvt. Ltd. | June 2026